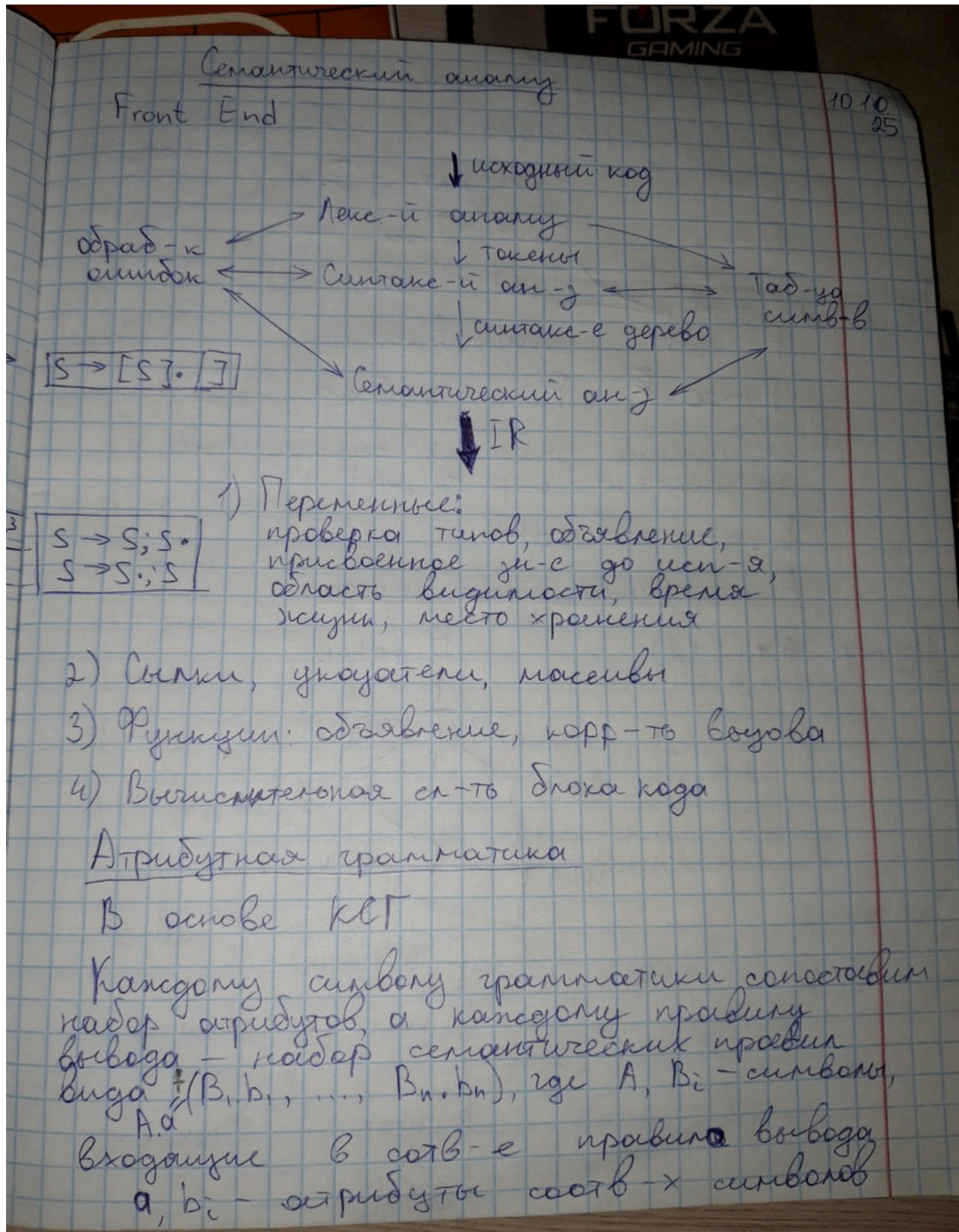


28. Примеры атрибутивных грамматик, не вычисляемых в ходе синтаксического анализа.

Атрибутивная грамматика:



Атрибутные грамматики

Возьмем произвольную КС-грамматику G и поставим в соответствие каждому грамматическому символу конечное (возможно пустое) множество параметров. Эти параметры будем называть *атрибутами*. Каждый из атрибутов имеет свою область изменения (*домен*). Каждому правилу вывода грамматики G сопоставим множество *семантических правил*, или *действий* – функциональных зависимостей вида $b := f(c_1, \dots, c_k)$, где $b, c_1, \dots, c_k$ – атрибуты грамматических символов, входящих в правило вывода. Полученный объект называется *атрибутной грамматикой*.

Пример 1

	Правило вывода	Семантические правила
1	$S \rightarrow L$	$L.i := 0$
2	$L \rightarrow L_1 1$	$L_1.i := L.i + 1$
3	$L \rightarrow \lambda$	PRINT($L.i$)

Данная L-атрибутная грамматика основана на очень простой LR(1)-грамматике, однако ее нельзя транслировать в ходе LR-анализа, поскольку первая свертка всегда производится по правилу $L \rightarrow \lambda$, а значение атрибута $L.i$, т. е. количество единиц в строке, в этот момент неизвестно. В данном случае наиболее простое решение — преобразование атрибутной грамматики, а не переход к более сложным методам трансляции.

Пример 1: преобразование к S-атрибутной грамматике

	Правило вывода	Семантические правила
1	$S \rightarrow L$	$L.i := 0$
2	$L \rightarrow L_1 1$	$L_1.i := L.i + 1$
3	$L \rightarrow \lambda$	$\text{PRINT}(L.i)$



	Правило вывода	Семантические правила
1	$S \rightarrow L$	$\text{PRINT}(L.s)$
2	$L \rightarrow L_1 1$	$L.s := L_1.s + 1$
3	$L \rightarrow \lambda$	$L.s = 0$

Пример 1: преобразование правила

Если правило $L \rightarrow L_1 1$ заменить на $L \rightarrow 1 L$, то можно оставить атрибут наследуемым и применить алгоритм маркирования грамматики

Правило вывода	Семантические правила	Действия в стеке
$S \rightarrow ML$	$L.i := M.s$	
$M \rightarrow \lambda$	$M.s = 0$	$val[top] = 0$
$L \rightarrow 1NL_1$	$N.i = L.i + 1, L_1.i := N.s$	
$N \rightarrow \lambda$	$N.s = N.i$	$val[top] = val[top - 2] + 1$
$L \rightarrow \lambda$	$\text{PRINT}(L.i)$	$\text{PRINT}(val[top - 1])$

Пример 2: разнонаправленный обход

Правило вывода	Семантические правила
$A \rightarrow L_1 L_2$	$L_1.i := f_1(A.i)$ $L_2.i := g_1(L_1.s)$ $A.s := h_1(L_2.s)$
$A \rightarrow R_1 R_2$	$R_2.i := f_2(A.i)$ $R_1.i := g_2(R_2.s)$ $A.s := h_2(R_1.s)$

Данная грамматика не является L-атрибутной. Для ее трансляции необходимо предварительно построить дерево вывода. Трансляция первого из правил требует стандартного обхода

сыновей узла A слева направо, а второго правила – обхода справа налево, несовместимого с синтаксическим анализом (та часть входной строки, в которую разворачивается R_2 , недоступна в момент применения правила $A \rightarrow R_1 R_2$).

Пример 2: разнонаправленный обход

Правило вывода	Семантические правила
$A \rightarrow L_1 L_2$	$L_1.i := f_1(A.i)$ $L_2.i := g_1(L_1.s)$ $A.s := h_1(L_2.s)$
$A \rightarrow R_1 R_2$	$R_2.i := f_2(A.i)$ $R_1.i := g_2(R_2.s)$ $A.s := h_2(R_1.s)$

Здесь применяется второй способ определения порядка выполнения семантических действий: для каждого правила вывода порядок выполнения действий свой и определяется на стадии

построения компилятора. На первом проходе строится дерево вывода, на втором вычисляются атрибуты при помощи набора рекурсивных функций для нетерминалов, похожих на функции нисходящего транслятора. Отличие в том, что синтаксический разбор уже выполнен, для каждого узла известны правило и упорядоченный список сыновей. Порядок обхода сыновей определяется только правилом вывода.

Пример 3: сложная зависимость

Правило вывода	Семантические правила
$S \rightarrow E$	$E.i := h_1(E.s)$ $S.t := E.t$
$E \rightarrow E_1 E_2$	$E.s := f_1(E_1.s, E_2.s)$ $E_1.i := g_1(E.i)$ $E_2.i := g_2(E.i)$ $E.t := f_2(E_1.t, E_2.t)$
$E \rightarrow x$	$E.s := x.s$ $E.t := h_2(E.i)$

Нетерминал E имеет два синтезируемых и один наследуемый атрибут. Видно, что синтезируемый атрибут $.s$ в узле всегда зависит только от тех же атрибутов сыновей узла. Наследуемый атрибут $.i$ в узле

зависит от того же атрибута родителя узла либо, если родитель – корень дерева, от атрибута $.s$ самого узла. Наконец, синтезируемый атрибут $.t$ зависит от тех же атрибутов сыновей, а в случае, когда сын является листом, – от атрибута $.i$ самого узла.



Пример 3: сложная зависимость

Правило вывода	Семантические правила
$S \rightarrow E$	$E.i := h_1(E.s)$ $S.t := E.t$
$E \rightarrow E_1 E_2$	$E.s := f_1(E_1.s, E_2.s)$ $E_1.i := g_1(E.i)$ $E_2.i := g_2(E.i)$ $E.t := f_2(E_1.t, E_2.t)$
$E \rightarrow x$	$E.s := x.s$ $E.t := h_2(E.i)$

Для вычисления
всех атрибутов нужно:

- 1) пройти по дереву вывода снизу вверх, вычисляя атрибут *.s*;
- 2) пройти сверху вниз, вычисляя атрибут *.i*;
- 3) пройти снизу вверх, вычисляя атрибут *.t*.

На практике

грамматика транслируется в два прохода: проводится синтаксический анализ с построением дерева и вычислением значений атрибута *.s*, а затем по дереву вычисляются значения атрибутов *.i* и *.t* (один стандартный обход в глубину).

Пример 3: сложная зависимость

Для чего нужна атрибутная грамматика именно с такой структурой семантических действий? Рассмотрим язык программирования, не требующий явного определения типов. При компиляции программы нужно произвести разбор выражения и установить тип выражения и типы всех его подвыражений на основании контекста, после чего внести информацию о типах в генерируемый код.

Для этого необходимо: 1) для каждого подвыражения определить множество допустимых типов (вычисляется снизу вверх по дереву – это синтезируемый атрибут $E.s$); 2) выбрать из этого множества единственный тип (это производится сверху вниз – фиксируется допустимый тип для всего выражения, а потом на основании этого уточняются типы подвыражений; выбранный тип представлен в примере наследуемым атрибутом $E.i$); 3) на основе выбранных типов снизу вверх по дереву генерируется код – синтезируемый атрибут $E.t$.

В теории компиляции не все атрибутные грамматики могут быть вычислены «на лету» одновременно с синтаксическим разбором (то есть за один проход). Это происходит, когда зависимости между атрибутами противоречат порядку обхода дерева вывода синтаксическим анализатором.

1. Грамматики с «несвоевременными» действиями

Это ситуации, когда семантическое действие должно быть выполнено в момент, когда необходимая для него информация еще не доступна анализатору.

- **Пример:** Грамматика для подсчета количества единиц в строке, где значение наследуемого атрибута передается в самое начало вывода.
 - Правила:
 1. $S \rightarrow L$ (семантика: $L.i := 0$)
 2. $L \rightarrow L_1 1$ (семантика: $L_1.i := L.i + 1$)
 3. $L \rightarrow \epsilon$ (семантика: $\text{print}(L.i)$)
 - **Почему не вычислим:** При LR-анализе первая свертка всегда происходит по аннулирующему правилу $L \rightarrow \epsilon$. В этот момент общее количество единиц в строке ($L.i$) еще не известно, так как символы строки еще не обработаны.

2. Не L-атрибутные грамматики (Разнонаправленные обходы)

Если грамматика не является L-атрибутной, это означает, что атрибуты могут зависеть от информации, находящейся «справа» от текущего узла, что невозможно вычислить при стандартном проходе слева направо.

- **Пример:** Правила вывода требуют разного порядка обхода сыновей для разных веток.
 - В одном правиле ($A \rightarrow L_1 L_2$) атрибуты вычисляются слева направо.
 - В другом правиле того же нетерминала ($A \rightarrow R_1 R_2$) наследуемый атрибут левого символа ($R_1.i$) зависит от синтезируемого атрибута правого символа ($R_2.s$).

- **Почему не вычислимо:** Для трансляции второго правила требуется обход справа налево, что несовместимо с синтаксическим анализом, так как правая часть входной строки (R_2) недоступна в момент обработки левой (R_1). Такие грамматики требуют предварительного построения полного дерева вывода.

3. Грамматики со сложными зависимостями (Многопроходные)

Некоторые грамматики требуют нескольких полных обходов дерева вывода (вверх-вниз-вверх) для вычисления всех значений.

- **Пример:** Определение типов в языках без явного описания переменных.
 - Процесс требует:
 1. Прохода **снизу вверх** для определения множества *допустимых* типов подвыражений.
 2. Прохода **сверху вниз** для *выбора* конкретного типа на основе контекста всего выражения.
 3. Повторного прохода **снизу вверх** для генерации кода на основе уже выбранных типов.
 - **Почему не вычислимо:** Синтаксический анализатор совершает только один проход. Такие зависимости атрибутов (синтезируемый $\rightarrow$ наследуемый $\rightarrow$ синтезируемый) технически невозможно разрешить за один такт построения дерева.

Вывод: Для обработки таких грамматик компиляторы используют «многопроходную» трансляцию: сначала строится полное дерево вывода (синтаксический анализ), а затем по нему запускаются рекурсивные функции для вычисления атрибутов в нужном порядке.